

COMPLETE CACHING DOCUMENTATION (BEGINNER → SENIOR / SYSTEM DESIGN LEVEL)

1. WHAT IS CACHING?

Caching is a **performance optimization technique** where frequently accessed or expensive-to-compute data is stored in a **faster storage layer** so future requests can be served quickly without recomputing or re-fetching from the primary source.

Golden Rule: Cache is an optimization, **never the source of truth**.

Primary source of truth = **Database**

2. WHY CACHING IS REQUIRED

Without caching: - Every request hits DB - Higher latency - DB overload - Poor scalability

With caching: - Faster response times - Reduced DB load - Better scalability - Cost optimization

3. CACHING LAYERS (MENTAL MODEL)

```
Client Cache (Browser / App)
    ↓
Server Cache (Redis / In-memory)
    ↓
Database Cache (Indexes / Buffer Pool)
    ↓
Database Storage
```

Each layer solves a **different problem**.

4. TYPES OF CACHING BY LAYER

4.1 CLIENT-SIDE CACHING

What it is: - Browser cache - HTTP cache headers - Service Workers - LocalStorage / IndexedDB

Use when: - Data is public or user-specific - Read-heavy - Not highly sensitive - Same response reused

Examples: - Product listing - Static pages - User profile

Do NOT use when: - Financial data - Real-time data - Highly sensitive data

Example (HTTP Cache):

Cache-Control: public, max-age=300

4.2 SERVER-SIDE CACHING (MOST IMPORTANT)

What it is: - Centralized shared cache (Redis) - Very fast (in-memory) - Controlled eviction and TTL

Use when: - Read-heavy APIs - Expensive DB queries - Aggregations - User/session data

Examples: - User profile - Dashboard data - Permissions - Search results

4.3 DATABASE-LEVEL CACHING

What it is: - DB buffer cache - Query plan cache - Indexes

Purpose: - Reduce disk I/O - Speed up queries

Important: - This is NOT application caching - Cannot replace Redis

5. CACHE WRITE STRATEGIES (CORE CONCEPT)

5.1 CACHE-ASIDE (LAZY LOADING) ★ DEFAULT STRATEGY

Read Flow: 1. App checks cache 2. Cache miss → DB 3. Data returned 4. Cache populated

Write Flow: 1. Write to DB 2. Invalidate cache (DELETE)

Characteristics: - DB is source of truth - Cache may be stale briefly - High availability - Simple and safe

Use for: - Most APIs - User data - Content systems

5.2 WRITE-THROUGH CACHE

Concept: - Every write goes to cache and DB synchronously

Characteristics: - Strong consistency - Slower writes - Cache is critical dependency

Use for: - Auth configs - Feature flags - Critical reference data

5.3 WRITE-BEHIND (WRITE-BACK) CACHE

Concept: - Write only to cache - DB updated asynchronously

Characteristics: - Very fast writes - Risk of data loss - Complex recovery

Use for: - Logs - Analytics - Metrics

6. TTL (TIME TO LIVE) STRATEGY

TTL defines **how long data lives in cache**.

Typical TTLs: | Data Type | TTL | |-----|-----| User profile | 5-15 min | Permissions | 1-5 min | Dashboard | 30-120 sec | Config | 10-60 min | Search results | 1-5 min |

Best Practices: - Short TTL for volatile data - Longer TTL for stable data - Add randomness to avoid stampede

7. CACHE INVALIDATION (HARDEST PROBLEM)

Best practice: - Invalidate cache on write - Do NOT update cache blindly

Why DELETE is preferred: - Avoid race conditions - Prevent partial updates - DB remains truth

Key versioning:

`user:v2:123`

8. CACHE STAMPEDE PROBLEM

Problem: - Cache expires - Many requests hit DB simultaneously

Solutions: - Randomized TTL - Mutex / locking - Early refresh

9. CACHING AND CAP THEOREM

CAP = Consistency, Availability, Partition tolerance

Cache-Aside → AP leaning

- High availability
- Eventual consistency

Write-Through → CP leaning

- Strong consistency
- Lower availability

Most systems choose AP.

10. MULTI-LAYER CACHING (REAL SYSTEMS)

Example: Product Page

Browser Cache → CDN → Redis → DB Index → DB

Each layer reduces load on the next.

11. WHEN TO USE WHICH CACHE (DECISION TABLE)

Data Type	Cache Layer
Static assets	Client
Public APIs	Client + Server
User profile	Server
Auth tokens	Server
Search results	Server
Transactions	DB only
Reference data	Client + Server

12. BEST OVERALL STRATEGY (FINAL VERDICT)

Cache-Aside + Redis + TTL + Invalidation

Why? - High availability - Scales well - DB remains truth - Easy to debug - Industry proven

13. INTERVIEW-READY SUMMARY

Caching improves performance by reducing load on the database. Cache-aside is the most commonly used strategy due to its simplicity and availability guarantees. Write-through is used when strong consistency is required, while write-behind is reserved for non-critical high-throughput systems. Caching decisions are driven by data criticality, read/write patterns, and CAP trade-offs.

14. GOLDEN RULES (REMEMBER THESE)

1. Cache is never the source of truth
 2. Invalidate on write
 3. Prefer delete over update
 4. TTL is mandatory
 5. Use Redis for shared cache
 6. Choose consistency vs availability consciously
-

 You now have **complete caching knowledge equivalent to a senior backend/system design engineer.**